Practical work – Classification of physical activities and gestures

Course: Applied Machine Learning for Real-World Data (AML)
Academic year: 2026–2027
Teachers: Andres Perez-Uribe (andres.perez-uribe@heig-vd.ch)
Assistant: Benoît Hohl (benoit.hohl@heig-vd.ch)

Context

In this practical work you will build the **complete machine-learning pipeline** that turns raw smartphone motion signals into a classifier able to recognise human activities and gestures. The data are recorded by **you**, using a smartphone running the **Phyphox** application (Android / iPhone).

The practical work runs over two sessions. Session 1 is dedicated to recording, visualisation, windowing, feature extraction, and dimensionality reduction; Session 2 is dedicated to classification, evaluation, and interpretation.

Goals

This laboratory will guide you through:

- using the **Phyphox** application to capture acceleration data;
- visualising and inspecting raw time series;
- splitting continuous signals into **windows**;
- extracting **time-domain and frequency-domain features**;
- exploring high-dimensional data with **PCA, t-SNE and UMAP**;
- training and comparing **KNN, Random Forest and an Artificial Neural Network (MLP)**;
- evaluating models honestly (train-validation-test split, confusion matrix,...) and **interpreting the errors**.

Session plan

Section 1 – from signal to features

1. Raw data recording with Phyphox.
2. Data visualisation (notebook sections 1).
3. Windowing (notebook section 2).
4. Feature extraction (notebook section 3).
5. Dimensionality reduction: PCA, t-SNE and UMAP (notebook section 5).

> **Note** This split by section is indicative and can be adapted to your pace. If you have already completed Section 1, we recommend moving on to the next section without waiting for week 2!

Section 2 – from features to decisions

1. Classification with KNN, Random Forest and an ANN (notebook section 6).
2. Evaluation and comparison (notebook section 7).
3. Interpretation and short written answers (notebook section 8).

AI use and academic integrity policy

- **The recordings must be yours.** Each class must correspond to data you actually collected with your own phone.
- **You might be asked to explain your work.** At the end of this practical work, you may be asked to explain any part of your code, justify a parameter (window length, overlap, K, tree depth, network size, perplexity…), reproduce the logic of your pipeline, and interpret the results obtained.
- **Your data make your results unique.** Two groups recording a "hammer" gesture differently will obtain different features; copy-pasted results will not match your own plots.

Activity catalogue

Record **at least two activities from each of the three families below**.

- **(a) Postures and locomotion:** for example: sit, stand, walk, walking up stairs, walking down stairs, elevator.
- **(b) Exercises:** for example: push-ups, jumping jacks, crunch.
- **(c) Tool use:** for example: hammer, screwdriver, hand saw.

> **Consistency matters.** Keep the phone in the same place (same pocket / same hand) and use the same phone across all recordings.

0) Raw data recording

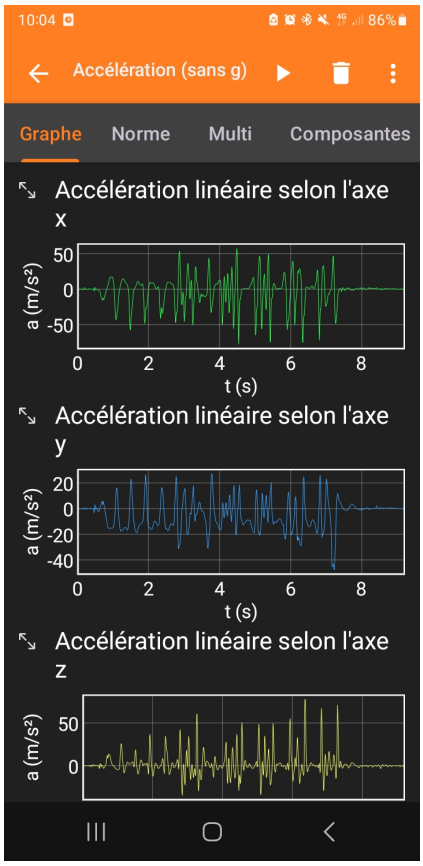


*Figure 1 – The Phyphox **Acceleration (without g)** experiment. The plots show the linear acceleration along the x, y and z axes (m/s^2) as a function of time (s).*

- For each activity:
- Open the Phyphox application and go to **Acceleration (without g)** (linear acceleration).
- Start the recording with the play button as you start to perform the activity. The recording should last **at least 1 minute**.
    - *Tip:* if a flight of stairs is too short for one whole minute, pause the recording after a set of stairs and restart it when you encounter other ones.
- Export the record:
    - open the options (three vertical dots) and choose **Export data**;
    - choose **CSV (Comma, decimal point)**;
    - share it using your favourite technique (mail, Telegram, WhatsApp, USB…);
    - unzip the archive and put the folder inside the `data` folder;
    - rename the folder with the name of the corresponding activity;
    - after exporting, press the trash button to remove the record before recording a new one.
- **Each activity is recorded and exported individually, in order to have one folder by activity.**

The `data/example_activity` folder provided in the repository shows the expected structure: a `Raw Data.csv` file plus a `meta` folder (`device.csv`, `time.csv`).

1) Data visualisation

In this part, you will plot the data that you collected. Open the notebook AML_phyphox_PW2627.ipynb and follow the instructions of the corresponding section in that notebook. Change activity_of_interest and measurement_of_interest in the code in order to have a good understanding of what your data

looks like. You should be able to perform it with your favorite IDE. Nevertheless, if you have any troubles, for example with packages, you can upload the folder of this PW on google drive and run DDM_lab.ipynb on google colab.

2) Windowing

Continuous recordings cannot be fed directly to a classifier. In the notebook you will split each recording of more than one minute into short **windows** (e.g. 1-2 s) with an optional **overlap**. Each window becomes one data point, and its class is the activity label. Think about the trade-off: short windows give more data points but noisier features; long windows give stable features but fewer samples.

3) Feature extraction

For each window you will extract a compact **feature vector** that summarises the signal of every channel. This is a very important step of the pipeline.

Each window contains **four channels**: the linear acceleration along the **x**, **y** and **z** axes, and the **absolute acceleration** (the norm of the three axes). Each feature is likely to be computed **for each channel**, so with k features you obtain a vector of $4 \times k$ values per window (e.g. 10 features → 40 values).

You can build your features in one of two ways:

- **Implement them yourself** with NumPy/SciPy. Start with simple time-domain features (mean, standard deviation, min, max, median, energy per channel), then add at least one **frequency-domain** feature (e.g. dominant frequency, spectral energy) computed from an FFT.
- **Use a dedicated library such as TSFEL** (Time Series Feature Extraction Library). It provides a catalogue of time-domain, statistical, spectral and fractal features, and extracts them window by window with a single call (`tsfel.get_features_by_domain()` and `tsfel.time_series_features_extractor()`). Install it with `pip install tsfel`. TSFEL can produce hundreds of features. Consider selecting a subset (by domain,or with feature selection) and justify your choice in the report.

Whichever approach you choose, your final feature set must include **both time-domain and frequency-domain features**.

4) Dimensionality reduction

Visualise your high-dimensional features in 2D using:

- **PCA** – linear, fast, tells you how much variance is preserved;
- **t-SNE** – non-linear, strong local structure preservation -> good for revealing clusters (tune the perplexity);
- **UMAP** – non-linear, preserves more global structure.

Compare what each method shows.

5) Classification

Train and compare three families of models:

- **KNN** – simple, interpretable; study the role of K;
- **Random Forest** – ensemble of trees; inspect feature importances;
- **ANN / MLP** – a small feed-forward neural network; tune the hidden layers.

Use a **train/validation** split strategy to evaluate and tune the models, and report accuracy and the **confusion matrix** for each model.

> **Note** Be careful when splitting into train and validation sets. Because consecutive windows come from the same continuous recording, they tend to be highly correlated (especially if they overlap). As a result, a purely random split can be misleading: very similar samples may end up on both sides.

6) Evaluation and interpretation

Produce a single comparison table (accuracy / macro-F1 per model) and a confusion matrix for your best model. Then answer, in your own words:

- Are there activities that are easy to classify? Difficult? Frequently confused? Why?
- Does the confusion matrix agree with what you observe on the PCA / t-SNE / UMAP plots?
- Which features were the most useful (Random Forest importances)?

Bonus) Feature selection

If you still have time and motivation, try to improve your pipeline by **adding more features** and perform **feature selection**.

1. **Add features.** Extend your feature set, for example with TSFEL, which can compute hundreds of temporal, statistical and spectral features per window and per channel.
2. **Feature selection.** A large feature set contains many redundant or useless features, which makes models slower and can hurt them. Implement a method to keep only the useful ones.

> **Note** Feature selection must use the **training set only**. If features are chosen with the validation data, the validation score is optimistic (data leakage). Fit the selection on the training windows, then apply it to the validation windows.

Deliverables

1. The completed notebook `AML_phyphox_PW2627.ipynb` (all `TODO` cells filled, all reflection questions answered in your own words).
2. Your `data` folder with the raw recordings for every activity.
3. We will organise a short **oral defence** during which you would be asked to explain your code, your parameters and/or your results.